Assignment 1: Bare-Metal Boot-Up of STM32F4 on QEMU

Github URL https://github.com/scholar-lab/stm32_qemu_baremetal_AngomUmakantaSingh
Student Angom Umakanta Singh
Entry No. 2025EEZ8518
Course Embedded Systems(ELL7283)

Introduction

Debian 13 Operating system is used for the experiment. As provided in the assignment instruction the necessary tools are installed by:

apt install qemu-system-arm gcc-arm-none-eabi gdb-multiarch make

The version of the tools are as below:

1. GNU gdb (Debian 16.3-1) 16.3
2. QEMU emulator version 10.0.7
3. arm-none-eabi-gcc 14.2.1

1 QEMU run and debug Commands

QEMU run with semihosting output

```
qemu-system-arm -M olimex-stm32-h405
-kernel firmware.elf
-nographic
-semihosting-config
enable=on,target=native.
```

To terminate the run `Ctrl + a, x`

QEMU run with debugging

```
 qemu-system-arm -M olimex-stm32-h405
-kernel firmware.elf
-S -gdb tcp::3333
-nographic
-semihosting-config
enable=on,target=native
```

The debugging is performed on another terminal using the command `gdb-multiarch firmware.elf`.

Debugging commands

Connect GDB to QEMU

`target remote :3333`

Reset

`monitor system_reset`

Set Breakpoint and execute

`break main` , continue

Show CPU registers

info registers

Inspect memory contents

`x/wx ADDRESS`

- `x` – examine memory
- `w` – word (32-bit)
- `x` – display in hexadecimal

2 Output

2.1 GDB Debug Evidence

The debugging command is as shown in Figure 1.

```
scholar@devian:~/Documents/embsys/assignment/stm32f4_qemu_mycode$ qemu-system-arm -M olimex-stm32-h405 \
  -kernel firmware.elf \
  -S -gdb tcp::3333 \
  -nographic \
  -semihosting-config enable=on,target=native
Boot OK
Data/BSS verified
Boot OK
Data/BSS verified
```

Figure 1: Starting QEMU in remote debugging mode using the GDB server.

The debugging output is shown in fig 2.

```
scholar@devian:~/Documents/embsys/assignment/stm32f4_qemu_mycode$ gdb-multiarch firmware.elf
GNU gdb (Debian 16.3-1) 16.3
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from firmware.elf...
(gdb) target remote :3333
Remote debugging using :3333
0x08000068 in main () at src/main.c:12
12          while (1);
(gdb) monitor system_reset
(gdb) break main
Breakpoint 1 at 0x800004a: file src/main.c, line 9.
(gdb) continue
Continuing.

Breakpoint 1, main () at src/main.c:9
9           sh_puts("Boot OK\r\n");
(gdb)
```

Figure 2: Debugging using gdb-multiarch.

2.2 Semihosting Observable output

The semihosting output for the given code is as shown in Fig 3.

```
scholar@devian:~/Documents/embsys/assignment/stm32f4_qemu_mycode$ qemu-system-arm \
  -M olimex-stm32-h405 \
  -kernel firmware.bin \
  -nographic \
  -semihosting-config enable=on,target=native
Boot OK
Data/BSS verified
```

Figure 3: Semihosting output of the binary.

2.3 SysTick Interrupt

SysTick Timer and Clock Implementation The program implements a simple digital clock using the SysTick timer available in the ARM Cortex-M4 processor. The SysTick is configured to generate an interrupt every 1 ms by setting the reload value based on the CPU clock frequency (168 MHz). Each interrupt increments a global counter in the `SysTick_Handler`. When the counter reaches 1000, it corresponds to one second, and the seconds variable is incremented. The program then updates minutes and hours when their respective limits (60 seconds and 60 minutes) are reached, forming a 24-hour clock. The current time is printed in the format `HH:MM:SS` using semihosting output functions. To avoid repeated printing within the same second, the program prints the time only when the second value changes. This demonstrates the use of periodic interrupts for timekeeping in embedded systems.

A clock is built using Systick interrupt. the output is as shown in Fig 4.

```
scholar@devian: ~/Documents/embsys/assignment/beta_systick
scholar@devian:~/Documents/embsys/assignment/beta_systick$ qemu-system-arm -M olimex-stm32-h405 -kernel firmware.elf -nographic -semihos
ting-config enable=on,target=native
Boot OK
Data/BSS verified
00:00:01
00:00:02
00:00:03
00:00:04
00:00:05
00:00:06
00:00:07
00:00:08
00:00:09
00:00:10
00:00:11
00:00:12
00:00:13
00:00:14
00:00:15
00:00:16
00:00:17
00:00:18
00:00:19
00:00:20
00:00:21
00:00:22
00:00:23
00:00:24
00:00:25
00:00:26
00:00:27
QEMU: Terminated
scholar@devian:~/Documents/embsys/assignment/beta_systick$
```

Figure 4: Output of clock implemented using SysTick.

3 Memory Map Used

The STM32 Cortex-M4 microcontroller uses a 32-bit address space in which different types of memory and peripherals are mapped to specific address regions. This organization is known as the memory map. It allows the processor to access program code, data, and hardware peripherals using normal memory read and write operations.

In this project, the firmware is stored in the Flash memory region starting at address `0x08000000`. The interrupt vector table is located at the beginning of this region. The first entry of the vector table contains the initial stack pointer, while the second entry contains the address of the `Reset_Handler`. When the processor starts, it loads the stack pointer from the first location and then jumps to the reset handler to begin program execution.

The SRAM region begins at address `0x20000000`. This memory is used during program execution to store global variables, static variables, the stack, and the heap. In this project, the stack pointer is initialized to the top of the available SRAM.

Peripheral devices such as timers, GPIO, and communication interfaces are mapped into the peripheral memory region starting at address `0x40000000`. These peripherals are accessed through memory-mapped registers.

System control components such as the Nested Vectored Interrupt Controller (NVIC), SysTick timer, and other debug registers are located in the system control space starting at address `0xE000E000`.

Region	**Start Address**	**Purpose**
Flash Memory	`0x08000000`	Program code and vector table
SRAM	`0x20000000`	Stack, heap, and variables
Peripheral Region	`0x40000000`	Hardware peripheral registers
System Control Space	`0xE000E000`	NVIC, SysTick, and system control

Table 1: Memory map of STM32 Cortex-M4

4 Vector table and Reset

The vector table is located at the beginning of the Flash memory and contains the initial stack pointer followed by the addresses of exception and interrupt handlers. During a reset event, the Cortex-M4 processor reads the first entry of the vector table and loads it into the Main Stack Pointer (MSP). The second entry contains the address of the `Reset_Handler`, which is loaded into the program counter, and execution begins from this function.

In this project, the vector table is defined in the `.isr_vector` section using assembly directives. The symbols `g_pfnVectors`, `Reset_Handler`, and `Default_Handler` are declared as global so that they are visible to the linker and can be referenced by other parts of the program. The directive `.type` informs the assembler that these symbols represent functions. The vector table is aligned to a 32-bit boundary using `.align 2`, which ensures proper word alignment required by the Cortex-M architecture.

Each entry of the vector table is defined using the `.word` directive, which stores a 32-bit address. The first entry sets the initial stack pointer to the top of RAM (`_estack`). The second entry specifies the address of the `Reset_Handler`, which performs system initialization and eventually calls the `main()` function. Subsequent entries correspond to core exception handlers such as NMI, HardFault, MemManage, BusFault, and UsageFault. In this implementation, these handlers point to a common `Default_Handler`. Reserved entries are set to zero, while system service interrupts such as SVCall, Debug Monitor, PendSV, and SysTick are also included in the table, with the SysTick interrupt mapped to `SysTick_Handler`.

The Cortex-M architecture defines a fixed set of system exception vectors followed by device-specific interrupt vectors. In total, the Cortex-M4 supports up to 240 external interrupts in addition to 16 system exception vectors, giving a maximum of 256 vector entries. In this project only the initial 16 system vectors are defined, which include the stack pointer, reset handler, core fault handlers, and system service interrupts.

5 Runtime Initialization

After the processor jumps to the `Reset_Handler`, the runtime initialization phase prepares the system before the application code begins execution. During this stage, the memory sections used by the program are properly initialized.

First, the initialized data section (`.data`) is copied from Flash memory, where it is stored after compilation, to SRAM so that global and static variables with predefined values are correctly set. Next, the uninitialized data section (`.bss`) is cleared by setting all its values to zero, ensuring that global and static variables without explicit initialization start with a defined value.

These steps establish a correct runtime environment for the program. Once the memory initialization is completed, control is transferred to the `main()` function, where the main application logic of the firmware begins execution.

6 Linker Script

The linker script defines how the compiled program is arranged in memory and specifies the locations where different sections of the program are placed. It also defines important symbols used during program startup.

The directive `ENTRY(Reset_Handler)` specifies that program execution should begin at the `Reset_Handler`. When the system starts, the processor loads the reset handler address from the vector table and begins execution from this entry point.

The `MEMORY` block defines the available memory regions in the system. In this project, two memory regions are defined. The Flash memory region begins at address `0x08000000` with a size of 1024 KB and is used to store program code and read-only data. The RAM region begins at address `0x20000000` with a size of 128 KB and is used for runtime data such as global variables, stack, and heap.

The symbol `_estack` is defined as the top of RAM using the expression `ORIGIN(RAM) + LENGTH(RAM)`. This value is used to initialize the stack pointer in the vector table.

The `SECTIONS` block describes how different parts of the program are placed into memory. The `.isr_vector` section is placed at the beginning of Flash memory and contains the interrupt vector table. The `KEEP` directive ensures that the vector table is not removed during linker optimization.

The `.text` section contains the program instructions and read-only data. It includes the compiled code (`.text*`) and constant data (`.rodata*`). The symbol `_etext` marks the end of this section.

The `.data` section stores initialized global and static variables. These variables are stored in Flash after compilation but must be copied to RAM during startup. The symbol `_sidata` holds the address in Flash where the initial data values are stored, while `_sdata` and `_edata` mark the start and end of the data section in RAM.

The `.bss` section contains uninitialized global and static variables. During runtime initialization, this section is cleared by setting all its values to zero. The symbols `_sbss` and `_ebss` define the start and end of this section in RAM.

Overall, the linker script ensures that the vector table and program code are stored in Flash memory, while runtime data sections such as `.data` and `.bss` are placed in RAM and properly initialized during system startup.

7 Boot Sequence Explanation

- When the STM32 Cortex-M4 processor is powered on or reset, it begins execution from the start of the memory map where the vector table is located.
- The processor reads the first entry of the vector table at address `0x08000000`. This value is loaded into the Main Stack Pointer (MSP) to initialize the stack.
- The second entry in the vector table contains the address of the `Reset_Handler`.
- The processor loads this address into the Program Counter (PC) and begins execution from the `Reset_Handler`.
- The `Reset_Handler` performs basic system initialization required before the main program starts.
- It initializes the data section by copying initialized global variables from Flash memory to SRAM.
- The uninitialized data section (`.bss`) is cleared by setting all values to zero.
- After completing memory initialization, the `Reset_Handler` calls the `main()` function.
- The `main()` function contains the primary application logic of the program.
- If the program finishes execution or encounters an unexpected interrupt, control may transfer to a default interrupt handler.

8 Build Reproducibility

To ensure build reproducibility, the binary is generated twice on the same system and the checksum is compared to check for any difference as shown in fig 5. The binary file is found to be same.

```
scholar@devian: ~/Documents/embsys/assignment/stm32f4_qemu_mycode
scholar@devian:~/Documents/embsys/assignment/stm32f4_qemu_mycode$ make clean
rm -f src/main.o startup/startup_stm32f4.o firmware.elf firmware.bin firmware.map
scholar@devian:~/Documents/embsys/assignment/stm32f4_qemu_mycode$ make
arm-none-eabi-gcc -mcpu=cortex-m4 -mthumb -O0 -g -Wall -ffreestanding -nostdlib -mfpu=fpv4-sp-d16 -mfloat-abi=hard -c src/main.c -o src/main.o
arm-none-eabi-gcc -mcpu=cortex-m4 -mthumb -O0 -g -Wall -ffreestanding -nostdlib -mfpu=fpv4-sp-d16 -mfloat-abi=hard -c startup/startup_stm32f4.s -o startup/startup_stm32f4.o
arm-none-eabi-gcc src/main.o startup/startup_stm32f4.o -T ld/linker.ld -nostdlib -Wl,-Map=firmware.map -o firmware.elf
arm-none-eabi-size firmware.elf
   text    data     bss     dec     hex filename
    224       4       4     232      e8 firmware.elf
arm-none-eabi-objcopy -O binary firmware.elf firmware.bin
scholar@devian:~/Documents/embsys/assignment/stm32f4_qemu_mycode$ sha256sum firmware.bin >build1
scholar@devian:~/Documents/embsys/assignment/stm32f4_qemu_mycode$ cat build1
acb032242ba10a452ac0a0fe6635cf690a29c703ed0bcdc2c0988c613ac0ff80  firmware.bin
scholar@devian:~/Documents/embsys/assignment/stm32f4_qemu_mycode$ make clean
rm -f src/main.o startup/startup_stm32f4.o firmware.elf firmware.bin firmware.map
scholar@devian:~/Documents/embsys/assignment/stm32f4_qemu_mycode$ make
arm-none-eabi-gcc -mcpu=cortex-m4 -mthumb -O0 -g -Wall -ffreestanding -nostdlib -mfpu=fpv4-sp-d16 -mfloat-abi=hard -c src/main.c -o src/main.o
arm-none-eabi-gcc -mcpu=cortex-m4 -mthumb -O0 -g -Wall -ffreestanding -nostdlib -mfpu=fpv4-sp-d16 -mfloat-abi=hard -c startup/startup_stm32f4.s -o startup/startup_stm32f4.o
arm-none-eabi-gcc src/main.o startup/startup_stm32f4.o -T ld/linker.ld -nostdlib -Wl,-Map=firmware.map -o firmware.elf
arm-none-eabi-size firmware.elf
   text    data     bss     dec     hex filename
    224       4       4     232      e8 firmware.elf
arm-none-eabi-objcopy -O binary firmware.elf firmware.bin
scholar@devian:~/Documents/embsys/assignment/stm32f4_qemu_mycode$ sha256sum firmware.bin >build2
scholar@devian:~/Documents/embsys/assignment/stm32f4_qemu_mycode$ cat build2
acb032242ba10a452ac0a0fe6635cf690a29c703ed0bcdc2c0988c613ac0ff80  firmware.bin
scholar@devian:~/Documents/embsys/assignment/stm32f4_qemu_mycode$ diff build1 build2
scholar@devian:~/Documents/embsys/assignment/stm32f4_qemu_mycode$
```

Figure 5: Checking the difference of two builds.